

# Smart Contract & Test Architecture Diagrams — Explained

---

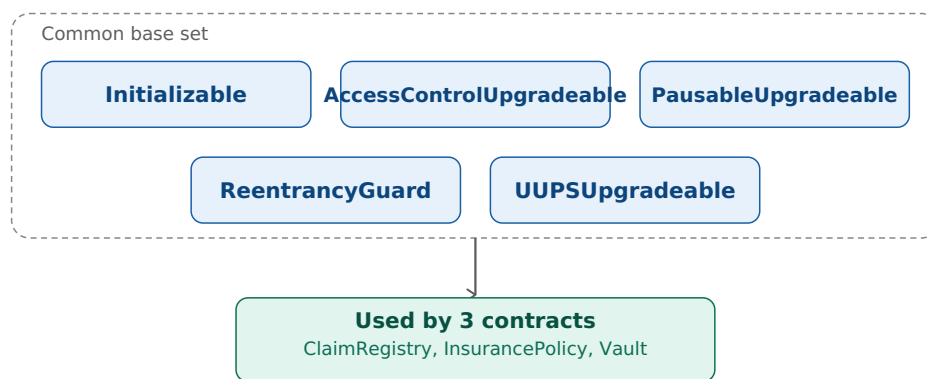
This document collects the nine reference diagrams built for the DICS v2 smart-contract layer and its test suite, each followed by a consistent five-part explanation: what the diagram is for, why it needed to exist at all, what it actually shows, what it's built from, and — the part most explanations skip — what specifically goes wrong in practice if a developer doesn't have this picture in their head.

**How to read the colors across this whole document:** color always groups things into a *category* (a shared base class, an entity, a governance mechanism) — it is never a severity scale, except in diagrams 6 and 9, where red specifically marks a revert/failure outcome and is called out as such at that point.

---

## 1. Contract inheritance, grouped by shared base classes

---



### Remaining contracts, each a distinct set

---

|                     |                                                      |
|---------------------|------------------------------------------------------|
| OracleAdapter       | Initializable, AccessControl, Pausable, UUPS, EIP712 |
| PriceOracle         | Initializable, AccessControl, UUPS                   |
| StableCoin          | plain ERC20, not upgradeable                         |
| DICSGovernanceToken | ERC20, ERC20Permit, ERC20Votes                       |
| DICSGovernor        | Governor plus five Governor extensions               |
| ClaimGasPaymaster   | BasePaymaster (EIP-4337)                             |

**Purpose.** Shows which OpenZeppelin (and account-abstraction) base contracts each of the 9 project contracts is actually built from.

**Why this diagram needed to exist.** Solidity resolves multiple inheritance through explicit `override(...)` lists, and getting that list wrong doesn't fail quietly — it either won't compile at all, or compiles but silently resolves to the wrong parent implementation. This project hit that exact problem more than once while building `DICSGovernor.sol`, where the correct override target genuinely differed between functions and even shifted between two supposedly identical dependency installs. A diagram that makes the shared base set visible at a glance is what turns "why won't this compile" into "oh, that's why."

**What it explains.** Three of the nine contracts — `ClaimRegistry`, `InsurancePolicy`, and `Vault` — share the exact same five-contract foundation. The other six each diverge in a specific, deliberate way: `OracleAdapter` swaps `ReentrancyGuard` for `EIP712Upgradeable`; `PriceOracle` drops both `Pausable` and `ReentrancyGuard` entirely; `StableCoin` isn't

upgradeable at all; and the governance/token/ paymaster contracts each sit in a completely different inheritance family.

**What it consists of.** A dashed container grouping the five shared base contracts, one summary box naming the three contracts that use all five together, and a compact reference list for the remaining six contracts' own distinct base sets.

**What goes wrong without it.** A developer copy-pasting an inheritance list from one contract to another — a very natural shortcut — could add `ReentrancyGuardUpgradeable` (which does not exist at all in OpenZeppelin v5, a real bug this project hit directly), drop `UUPSUpgradeable` by accident (silently making the contract non-upgradeable forever, since that mistake only surfaces the day someone actually tries to upgrade it), or misjudge which functions need an `override` list in the first place.

---

## 2. InsurancePolicy.sol storage layout

---

### Inherited base contracts

AccessControlUpgradeable, PausableUpgradeable, Initializable  
each keep their own storage at a separate ERC-7201 hashed location

### InsurancePolicy's own sequential storage

|            |                           |
|------------|---------------------------|
| Slot 0     | _policies mapping         |
| Slot 1     | _exists mapping           |
| Slot 2     | premiumToken              |
| Slot 3     | premiumTreasury           |
| Slot 4     | premiumGracePeriodSeconds |
| Slot 5     | premiumTerms mapping      |
| Slot 6     | premiumPaidUntil mapping  |
| Slot 7     | policyTemplates mapping   |
| Slot 8     | nextPolicyId              |
| Slots 9-48 | __gap[40] reserved        |

**Purpose.** Shows exactly which storage slot each of `InsurancePolicy.sol`'s own declared variables occupies, in the exact order they were declared, and where the reserved buffer for future additions sits.

**Why this diagram needed to exist.** Upgradeable contracts are uniquely fragile around storage in a way most developers coming from ordinary Solidity, or from any Web2 backend, have never had to think about: two deployments of the *same logical contract* — the old version and the new one — must agree on what every storage slot *means*, because the proxy's storage physically persists across the upgrade untouched. OpenZeppelin v5 makes this subtler still by moving each *inherited* contract's own storage into a separately hashed location (ERC-7201), which is exactly the distinction this diagram draws between the dashed "inherited" box and the solid "own storage" box below it.

**What it explains.** Slots 0–1 predate the premium module; slots 2–6 were added when premium billing was built; slots 7–8 were added later still, when the self-service policy catalog was built — each addition correctly *appended* rather than inserted, and each addition correctly shrank the reserved `__gap` by exactly the number of new slots it consumed ( $47 \rightarrow 42 \rightarrow 40$ , as documented in the contract's own history).

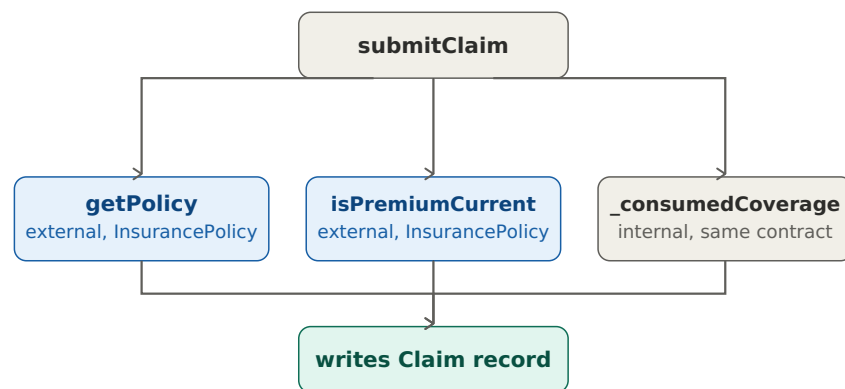
**What it consists of.** A dashed note explaining that base-contract storage lives elsewhere entirely, a solid container for the contract's own sequential slots, one row per declared variable color-grouped by which feature introduced it, and a final reserved block.

**What goes wrong without it.** If a future change inserted a new variable in the *middle* of this list instead of appending it before the gap, every variable declared after it would silently shift to a different slot number the moment the upgrade went live — meaning the new implementation code would read and write the *wrong* variable's data for every single existing policy, with no error, no revert, and no warning. This is the single most catastrophic and hardest-to-detect class of bug in upgradeable contracts, and the entire reason `__gap` and "always append, never insert" exist as a discipline in the first place.

---

### 3. `ClaimRegistry.submitClaim` call graph

---



**Purpose.** Shows exactly what `submitClaim` calls internally, and — critically — distinguishes calls that stay inside `ClaimRegistry` from calls that cross into a different deployed contract.

**Why this diagram needed to exist.** Reading a function's source code top to bottom doesn't make this distinction visually obvious: `getPolicy(...)` and `_consumedCoverage(...)` look almost identical as lines of code, but one of them leaves this contract's own execution context entirely and the other never does. That difference has real consequences — an external call can fail for reasons entirely outside this contract's control (the target is paused, the target was upgraded, the target simply reverts), while an internal call cannot.

**What it explains.** `submitClaim` makes exactly two external, read-only calls into `InsurancePolicy` (`getPolicy`, `isPremiumCurrent`) and one internal call within `ClaimRegistry` itself (`_consumedCoverage`), before it ever writes anything to storage.

**What it consists of.** One entry node, three called-function nodes each explicitly labeled internal or external and naming which contract they belong to, and a final node representing the actual state write that only happens after every check above it has passed.

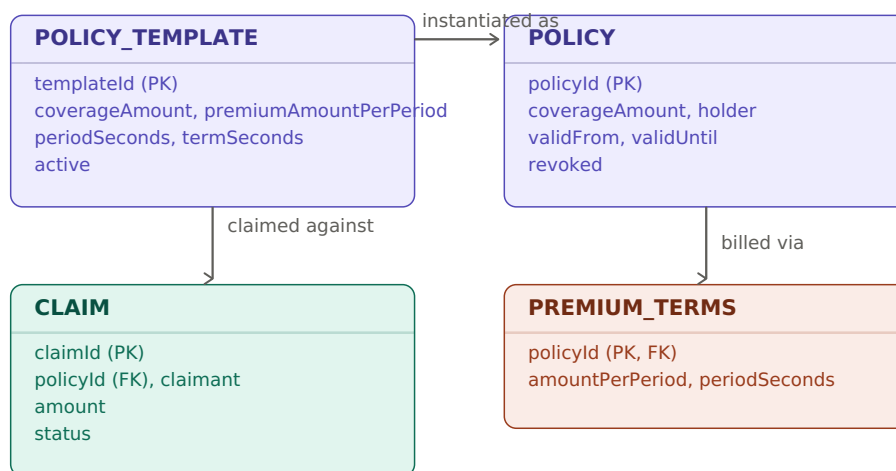
**What goes wrong without it.** A developer who doesn't internalize that two of these three calls are external might not realize that pausing or upgrading `InsurancePolicy` changes

what `submitClaim` does on `ClaimRegistry` — a cross-contract dependency that's easy to miss if you only ever read `ClaimRegistry.sol` in isolation. It also matters for ordering: external calls are exactly the kind of operation that belongs *after* internal state checks, not before — the classic checks-effects-interactions pattern this diagram makes visually concrete rather than abstract.

---

## 4. Policy, premium, and claim entity relationships

---



**Purpose.** Shows the on-chain data model on its own — which records exist and how they reference each other — independent of any single function that happens to touch them.

**Why this diagram needed to exist.** Without seeing the data model in one place, a developer tends to reconstruct it piecemeal from whatever function they're currently reading, and that reconstruction is easy to get subtly wrong — especially around cardinality (is this a one-to-one or one-to-many relationship?), which no amount of reading `submitClaim`'s code alone will make obvious.

**What it explains.** A `PolicyTemplate` is a catalog entry that gets copied into a brand-new `Policy` record at subscription time (not referenced by pointer — copied); every `Policy` has exactly one `PremiumTerms` record; and every `Policy` can have *many* `Claim` records over its lifetime, not just one.

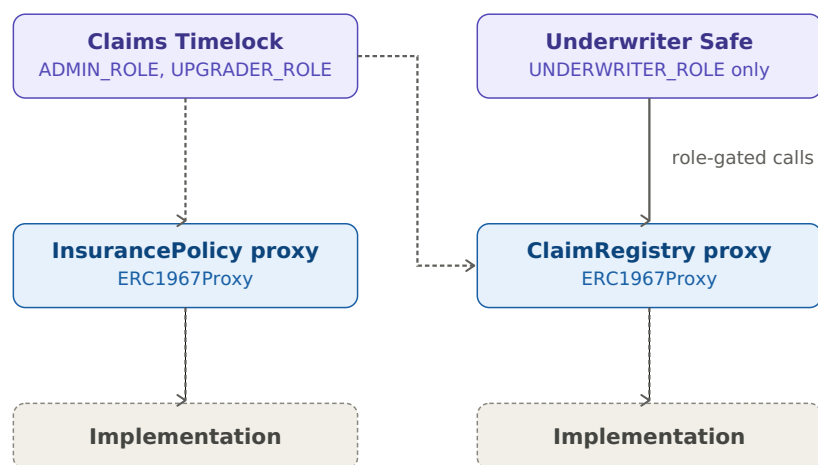
**What it consists of.** Four entity boxes, each listing its primary and foreign keys plus its other fields, connected by labeled relationship arrows.

**What goes wrong without it.** The most likely real mistake is assuming a policy can only ever have one active claim — which would lead to a bug (or an incorrect test) where filing a second, entirely legitimate claim against a policy that already has one seems like it should fail, when the actual system is explicitly designed to allow multiple claims against the same policy as long as their combined total stays within coverage. A second common mistake: trying to read premium information directly off the `Policy` struct, not realizing it's deliberately a separate record.

---

## 5. Deployment and ownership — who controls what

---



**Purpose.** Shows who actually controls what in the deployed system, distinguishing *upgrade authority* (the power to change a contract's logic entirely) from *operational role authority* (the power to call specific already-existing functions).

**Why this diagram needed to exist.** This project has two genuinely separate governance mechanisms now: a Timelock that can upgrade both proxies, and a Safe multisig that can approve or reject claims but has no upgrade power at all. Conflating "who can approve a claim" with "who can rewrite the contract's entire logic" is exactly the kind of mistake that turns a routine access-control decision into a real security incident.

**What it explains.** The Claims Timelock reaches both proxies with *dashed* upgrade-authority arrows and touches neither contract's actual business logic directly; the Underwriter Safe reaches only `ClaimRegistry`, with a *solid* arrow representing real, immediate function calls (`setClaimStatus`, `payoutClaim`); and each proxy delegates its logic to its own separate implementation contract.

**What it consists of.** Two governance boxes at the top, two proxy boxes in the middle, two implementation boxes at the bottom, dashed lines for upgrade authority, and one solid line for the role-gated operational call path.

**What goes wrong without it.** Someone provisioning access — granting a role, adding a new Safe owner — could reasonably but wrongly assume that adding themselves as a Safe owner also gives them any upgrade power, or that Timelock membership implies the ability to approve claims. Neither is true, and this diagram is what makes that structurally obvious rather than something you have to already know.

---

## 6. Test coverage matrix

---

| File                       | Happy path | Access control | Edge cases | Upgrade safety |
|----------------------------|------------|----------------|------------|----------------|
| InsurancePolicy.t.sol      | ●          | ●              | ●          | ○              |
| OracleAdapter.t.sol        | ●          | ●              | ●          | ○              |
| ClaimRegistryUpgrade.t.sol | ●          | ●              | ●          | ●              |
| Premium.t.sol              | ●          | ●              | ●          | ○              |
| PolicyCatalog.t.sol        | ●          | ○              | ●          | ○              |
| Vault.t.sol                | ●          | ●              | ●          | ○              |
| Governance.t.sol           | ●          | ●              | ●          | ○              |

● covered
○ not covered / not applicable

**Purpose.** Gives an at-a-glance view of which *categories* of behavior are actually tested in each of the 7 Foundry test files, rather than just a pass/fail count.

**Why this diagram needed to exist.** Test suites grow file by file, function by function, over the life of a project. Without a consolidated view like this, it's genuinely easy for a team to believe "we have good test coverage" purely because the total test count is high, without anyone having actually checked whether the tests that exist cover the categories that matter — access control and upgrade safety being the two categories where a gap is most expensive.

**What it explains.** Every file covers its own happy path. Most cover access control and edge cases too. But only one file in the entire suite — `ClaimRegistryUpgrade.t.sol` — actually exercises upgrade safety at all, and `PolicyCatalog.t.sol`'s "no" on access control is correct as designed: `subscribeToPolicy` is deliberately open to anyone, so there's no access restriction to test there in the first place.

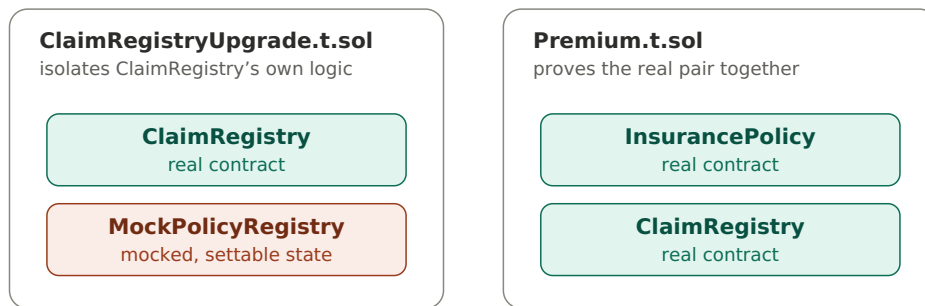
**What it consists of.** A 7-row by 4-column grid, filled dots meaning "covered," outlined dots meaning "not covered or not applicable," row labels naming each test file, and column headers naming each category.

**What goes wrong without it.** A team could ship a real change to, say, `Vault.sol`'s upgrade path with full confidence because "all tests pass," without ever noticing that upgrade safety is tested for exactly one contract in this entire project — a gap invisible from reading any single test file, and only visible once every file's coverage is placed side by side like this.

---

## 7. Mock versus real dependencies — the same choice, made two different ways

---



**Purpose.** Shows, for two specific test files, whether each contract they depend on is the real thing or a hand-built mock — and makes the point that this project made that choice *differently* for each file, on purpose.

**Why this diagram needed to exist.** Developers new to testing often treat "mock it" and "use the real thing" as a single global policy — either you mock everything for speed and isolation, or you use real dependencies for maximum confidence. Neither instinct is right on its own; the correct choice depends entirely on what a given test is actually trying to prove.

**What it explains.** `ClaimRegistryUpgrade.t.sol` mocks `InsurancePolicy` specifically so it can test `ClaimRegistry`'s own logic — rate limiting, upgrade safety, coverage math — without that logic being entangled with a second contract's behavior. `Premium.t.sol` does the opposite on purpose: it uses the real pair together specifically *because* the thing being tested — does a lapsed premium on `InsurancePolicy` actually block a claim on `ClaimRegistry` — only exists at the boundary between the two real contracts, and a mock would test nothing meaningful.

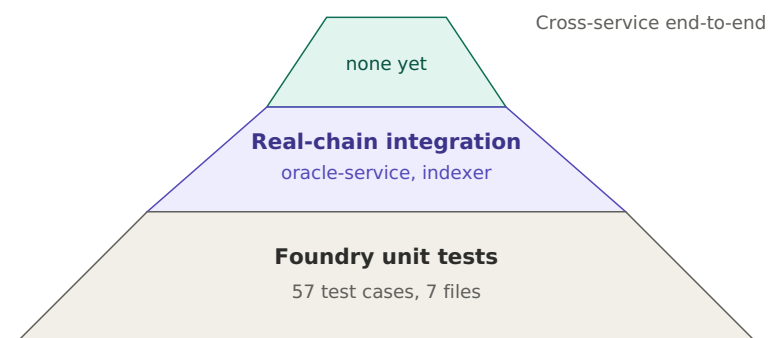
**What it consists of.** Two side-by-side containers, one per test file, each listing its dependencies marked as real or mocked.

**What goes wrong without it.** A developer might apply "always mock for unit tests" as a blanket rule and end up mocking away the exact cross-contract interaction that was the actual point of a test like `Premium.t.sol` — producing a test that passes reliably while proving nothing about whether the real integration actually works.

---

## 8. The shape of this project's test coverage

---



**Purpose.** Shows the actual *shape* of this project's test confidence across three tiers, using this project's real counts rather than a generic textbook pyramid.

**Why this diagram needed to exist.** A large raw test count feels reassuring, but it's easy to mistake "57 tests pass" for comprehensive confidence when almost all of that number sits in exactly one tier.

**What it explains.** 57 Foundry unit tests form a genuinely strong base. A much smaller number of tests actually run against a real Anvil chain — and only in two services, `oracle-service` and the indexer. Zero tests exist anywhere in this project that exercise the full, multi-service stack together end to end.

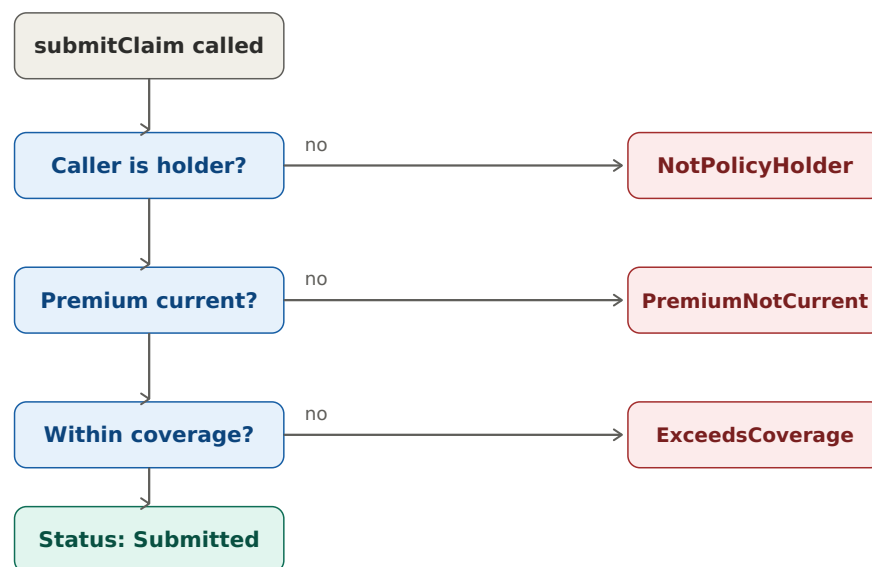
**What it consists of.** Three stacked trapezoid tiers, widest at the base, each labeled with what it tests and how many tests actually exist at that tier — including the top tier, labeled honestly as "none yet" rather than omitted.

**What goes wrong without it.** A reviewer could look at "57 passing tests" and reasonably but wrongly conclude the system as a whole is well-verified, when in reality no test in this project has ever proven that, for example, the Go notification service and the Java underwriter service agree with each other against the same real chain state at the same time — the exact kind of gap this pyramid's empty top tier is deliberately drawn to surface, not hide.

---

## 9. `submitClaim`'s revert conditions, in order

---



**Purpose.** Shows the exact order and content of the checks a real `submitClaim` call has to pass, and which specific, named error comes back at each possible failure point.

**Why this diagram needed to exist.** A list of `require` / `revert` statements scattered through a function's source doesn't make their *order* visually obvious, and order genuinely matters here — both for gas (cheaper checks generally belong earlier) and for correctness (a later check can safely assume an earlier one already passed, which isn't obvious unless the order itself is visible).

**What it explains.** The caller-is-holder check runs first, the premium-current check second,

and the coverage check third — each with its own distinct, named revert reason, only reaching the final "Status: Submitted" write once all three have passed.

**What it consists of.** A chain of decision boxes running down the left, each with a "no" branch to its own specifically named revert box on the right, ending in one success box at the bottom.

**What goes wrong without it.** A developer writing a new caller against this contract — or a new test — might not know *which* specific check will fire first for a given invalid input, leading to a test that asserts the wrong revert reason, or a misdiagnosed failure where the real cause (say, a lapsed premium) gets mistaken for a different one (say, insufficient coverage) simply because the actual evaluation order wasn't visible.